МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность ПО-13

Выполнил:
М.А.Шумило
студент группы
ПО-13
Проверил:
А.Д.Кулик
13.03.2026

Брест 2026

Цель работы: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

Задание 1.

3) Проект «Бургер-закусочная». Реализовать возможность формирования заказа из определенных позиций (тип бургера (веганский, куриный и т.д.)), напиток (холодный – пепси, кока-кола и т.д.; горячий – кофе, чай и т.д.), тип упаковки – с собой, на месте. Должна формироваться итоговая стоимость заказа.

Выполнение:

Код программы:

```
from abc import ABC, abstractmethod

class Burger:
    def __init__(self, name: str, price: float):
        self.name = name
        self.price = price

class Drink:
    def __init__(self, name: str, price: float, is_hot: bool):
        self.name = name
        self.price = price
        self.is_hot = is_hot

class Packing:
    def __init__(self, name: str, extra_cost: float = 0.0):
        self.name = name
        self.extra_cost = extra_cost

class Order:
    def __init__(self):
        self.burger: Burger | None = None
        self.drink: Drink | None = None
        self.packing: Packing | None = None

    @property
    def total_price(self) -> float:
        total = 0.0
        if self.burger:
            total += self.burger.price
        if self.drink:
            total += self.drink.price
        if self.packing:
            total += self.packing.extra_cost
        return total

    def __str__(self) -> str:
        lines = ["Ваш заказ:"]
        if self.burger:
            lines.append(f"Бургер: {self.burger.name} (цена:
{self.burger.price})")
        if self.drink:
            lines.append(f"Напиток: {self.drink.name} (цена:
{self.drink.price})")
        if self.packing:
            lines.append(
                f"Упаковка: {self.packing.name} (доплата:
{self.packing.extra_cost})"
            )
```

```
        lines.append(f"Итого: {self.total_price}")
        return "\n".join(lines)

class OrderBuilder(ABC):
    def __init__(self):
        self._order = Order()

    @abstractmethod
    def build_burger(self, burger_type: int) -> None:
        pass

    @abstractmethod
    def build_drink(self, drink_type: int) -> None:
        pass

    @abstractmethod
    def build_packing(self, packing_type: int) -> None:
        pass

    def get(self) -> Order:
        return self._order

class SimpleOrderBuilder(OrderBuilder):
    def build_burger(self, burger_type: int) -> None:
        if burger_type == 1:
            self._order.burger = Burger("Веганский бургер", 10.0)
        elif burger_type == 2:
            self._order.burger = Burger("Куриный бургер", 12.0)
        elif burger_type == 3:
            self._order.burger = Burger("Говяжий бургер", 15.0)
        else:
            raise ValueError("Неизвестный тип бургера")

    def build_drink(self, drink_type: int) -> None:
        if drink_type == 1:
            self._order.drink = Drink("Пепси", 5.0, is_hot=False)
        elif drink_type == 2:
            self._order.drink = Drink("Кока-кола", 5.0, is_hot=False)
        elif drink_type == 3:
            self._order.drink = Drink("Кофе", 6.0, is_hot=True)
        elif drink_type == 4:
            self._order.drink = Drink("Чай", 4.0, is_hot=True)
        else:
            raise ValueError("Неизвестный тип напитка")

    def build_packing(self, packing_type: int) -> None:
        if packing_type == 1:
            self._order.packing = Packing("С собой", extra_cost=1.0)
        elif packing_type == 2:
            self._order.packing = Packing("На месте", extra_cost=0.0)
        else:
            raise ValueError("Неизвестный тип упаковки")


print("Бургер-закусочная")
while True:
    print("\nСоздание нового заказа...")
    builder = SimpleOrderBuilder()
    print("Выберите бургер: 1 - веганский, 2 - куриный, 3 - говяжий")
    burger_t = int(input("Ваш выбор: "))
    builder.build_burger(burger_t)
    print("Выберите напиток: 1 - пепси, 2 - кока-кола, 3 - кофе, 4 - чай")
    drink_t = int(input("Ваш выбор: "))
    builder.build_drink(drink_t)
```

```
    print("Выберите упаковку: 1 - с собой, 2 - на месте")
    packing_t = int(input("Ваш выбор: "))
    builder.build_packing(packing_t)

    order = builder.get()
    print("\n" + str(order))

    print("\nХотите сделать ещё один заказ? (y/n)")
    again = input("Ваш выбор: ").strip().lower()

    if again != "y":
        break
```

Рисунки с результатами работы программы

```
Создание нового заказа...
Выберите бургер: 1 - веганский, 2 - куриный, 3 - говяжий
Ваш выбор: 2
Выберите напиток: 1 - пепси, 2 - кока-кола, 3 - кофе, 4 - чай
Ваш выбор: 4
Выберите упаковку: 1 - с собой, 2 - на месте
Ваш выбор: 1

Ваш заказ:
Бургер: Куриный бургер (цена: 12.0)
Напиток: Чай (цена: 4.0)
Упаковка: С собой (доплата: 1.0)
Итого: 17.0

Хотите сделать ещё один заказ? (y/n)
Ваш выбор:
```

Задание 2. Третья группа заданий (поведенческий паттерн)

3) Проект «IT-компания». В проекте должен быть реализован класс «Сотрудник» с субординацией (т.е. должна быть возможность определения кому подчиняется сотрудник и кто находится в его подчинении). Для каждого сотрудника помимо сведений о субординации хранятся другие данные (ФИО, отдел, должность, зарплата). Предусмотреть возможность удаления и добавления сотрудника.

Задание 3. Вторая группа заданий (структурный паттерн)

3) Проект «Расчет зарплаты». Для задания, указанного во втором пункте («IT-компания») реализовать расчет зарплаты с выводом полного отчета. Порядок вывода сотрудников в отчете – по старшинству для каждого отдела.

Выполнение:

Код программы:

```
class Employee:
    def __init__(self, name, department, position, salary):
        self.name = name
        self.department = department
        self.position = position
        self.salary = salary
        self.subordinates = []
        self.boss = None

    def add_subordinate(self, employee):
```

```
        self.subordinates.append(employee)
        employee.boss = self

    def remove_subordinate(self, employee):
        if employee in self.subordinates:
            self.subordinates.remove(employee)
            employee.boss = None

    def info(self, indent=0):
        print(" " * indent + f"{self.position}: {self.name} | Отдел:
{self.department} | Зарплата: {self.salary}")
        for sub in self.subordinates:
            sub.info(indent + 4)

    def get_boss(self):
        return self.boss

    def get_subordinates(self):
        return self.subordinates

class EmployeeIterator:
    def __init__(self, root):
        self.stack = [(root, 0)]

    def __iter__(self):
        return self

    def __next__(self):
        if not self.stack:
            raise StopIteration

        current, level = self.stack.pop()


        for sub in reversed(current.subordinates):
            self.stack.append((sub, level + 1))

        return current, level

def generate_salary_report(root_employee):
    iterator = EmployeeIterator(root_employee)

    printed_departments = set()
    total_salary = 0

    print("\nОТЧЁТ ПО ЗАРПЛАТАМ")

    for emp, level in iterator:

        if emp.department not in printed_departments:
            print(f"\nОтдел: {emp.department}")
            printed_departments.add(emp.department)

        indent = "    " * (level + 1)

        print(f"{indent}{emp.position}: {emp.name} — {emp.salary} руб.")

        total_salary += emp.salary

    print("\nИТОГО ЗАТРАТ НА ЗАРПЛАТЫ:", total_salary, "руб.")



def find_employee_by_name(root, name):
```

```
    for emp in EmployeeIterator(root):
        if emp.name.lower() == name.lower():
            return emp
    return None

def interactive_menu(root):
    while True:
        print("\nМЕНЮ УПРАВЛЕНИЯ КОМПАНИЕЙ")
        print("1 — Показать структуру компании")
        print("2 — Добавить сотрудника")
        print("3 — Удалить сотрудника")
        print("4 — Показать отчёт по зарплатам")
        print("0 — Выход")
        choice = input("Ваш выбор: ").strip()

        if choice == "1":
            print("\nСТРУКТУРА КОМПАНИИ:")
            root.info()

        elif choice == "2":
            print("\nДобавление сотрудника")
            name = input("ФИО: ")
            dept = input("Отдел: ")
            pos = input("Должность: ")
            salary = float(input("Зарплата: "))
            boss_name = input("Кому подчиняется (ФИО): ")

            boss = find_employee_by_name(root, boss_name)
            if not boss:
                print("Ошибка: начальник не найден.")
                continue

            new_emp = Employee(name, dept, pos, salary)
            boss.add_subordinate(new_emp)
            print(f"Сотрудник {name} добавлен под руководителя {boss.name}.")

        elif choice == "3":
            print("\nУдаление сотрудника")
            name = input("Введите ФИО сотрудника: ")
            emp = find_employee_by_name(root, name)

            if not emp:
                print("Сотрудник не найден.")
                continue

            if emp is root:
                print("Нельзя удалить директора!")
                continue

            boss = emp.get_boss()
            boss.remove_subordinate(emp)
            print(f"Сотрудник {name} удалён.")

        elif choice == "4":
            generate_salary_report(root)

        elif choice == "0":
            print("Выход из программы.")
            break

        else:
            print("Некорректный ввод. Попробуйте снова.")

if __name__ == "__main__":
```

```
    director = Employee("Иванов И.И.", "Управление", "Директор", 200000)
    head_dev = Employee("Петров П.П.", "Разработка", "Руководитель отдела",
150000)
    dev1 = Employee("Сидоров С.С.", "Разработчик", "Разработчик", 100000)
    dev2 = Employee("Кузнецов К.К.", "Разработчик", "Разработчик", 95000)
    hr = Employee("Смирнова А.А.", "HR", "HR-менеджер", 80000)


    director.add_subordinate(head_dev)
    director.add_subordinate(hr)
    head_dev.add_subordinate(dev1)
    head_dev.add_subordinate(dev2)

    interactive_menu(director)
```

Рисунки с результатами работы программы

```
ОТЧЁТ ПО ЗАРПЛАТАМ

Отдел: Управление
    Директор: Иванов И.И. — 200000 руб.

Отдел: Разработка
        Руководитель отдела: Петров П.П. — 150000 руб.

Отдел: Разработчик
            Разработчик: Сидоров С.С. — 100000 руб.
            Разработчик: Кузнецов К.К. — 95000 руб.

Отдел: HR
        HR-менеджер: Смирнова А.А. — 80000 руб.

ИТОГО ЗАТРАТ НА ЗАРПЛАТЫ: 625000 руб.

МЕНЮ УПРАВЛЕНИЯ КОМПАНИЕЙ
1 — Показать структуру компании

Отдел: Управление
    Директор: Иванов И.И. — 200000 руб.

Отдел: Разработка
        Руководитель отдела: Петров П.П. — 150000 руб.

Отдел: Разработчик
            Разработчик: Сидоров С.С. — 100000 руб.
            Разработчик: Кузнецов К.К. — 95000 руб.
```

Вывод: приобрел навыки применения паттернов проектирования при решении практических задач с использованием языка Python.